

R-project, Python y visualización de información

Scott, J. (2022). *R-project, Python y visualización de información*
[Apunte Docente]. Universidad Andrés Bello, Santiago.

Tabla de contenido

1. INTRODUCCIÓN A R-PROJECT Y PYTHON	2
1.1 ¿Qué es R-project y Python?	2
1.1.1 ¿Cuáles son las principales librerías de R y Python?	3
1.1.2 Entorno IDE (entorno de desarrollo integrador) para R-Studio	3
1.1.3 Primeros pasos en Python	4
1.1.4 Cómo instalar librerías en RStudio y Pycharm	4
1.2 Sintaxis del lenguaje RStudio y Python	4
1.3 Principales objetos de estudio para RStudio y Python	6
1.4 Librerías de análisis de datos	8
1.4.1 Librerías disponibles para R	9
1.4.2 Librerías disponibles para Python	15
2. VISUALIZACIÓN Y ANALÍTICA DE DATOS	19
2.1 Carga y visualización de datos	19
2.1.1 Visualización de datos usando data frame de la librería Pandas de Python	20
2.2 Reemplazos de valores y mutación de datos	23
2.3 Ordenamiento y filtros	25
2.4 Gráfica	26
2.5 Modelos matemáticos	29
2.6 Introducción a los modelos de inteligencia artificial (AI)	31
Referencias bibliográficas	32

1. INTRODUCCIÓN A R-PROJECT Y PYTHON

1.1 ¿Qué es R-proyect y Python?

Son lenguajes orientados interpretados, es decir, que no requieren ser compilados para ser ejecutados en una máquina computacional.

A pesar de ser lenguajes orientados a objetos, son de sentencias sencillas y sintaxis simples e intuitivas, para las que no se requiere ser un programador experimentado para aprenderlos y aplicarlos.

Este tipo de lenguajes son altamente demandados para trabajar funciones matemáticas complejas, las que son aplicables a la analítica y minería de datos, así como a las estadísticas y aprendizaje de máquina. La razón es que disponen de una amplia gama de bibliotecas desarrollada por una comunidad colaborativa de código abierto, la que permite la simplificación en la generación de código.

Tanto R como Python pueden conectarse a distintas bases de datos, como también conjuntos de datos, set de datos, en tiempo real que facilita el estudio de la composición de los datos, como también su facilidad para hacer gráficas de alta calidad.

Hay muchos ejemplos que se pueden lograr con el lenguaje R y Python:

- Lectura de set de datos en distintos formatos Excel, txt, con tablas de base de datos en muy pocas líneas de código.
- Análisis de datos mediante los distintos métodos de administración de matrices de datos.
- Aplicar metodologías de normalización de información en un conjunto de datos, para hacer minería de datos.
- Construir algoritmos de aprendizaje de máquina mediante método de reforzamiento, búsqueda de patrones, neuronales sobre un conjunto de información.
- Aplicaciones desde graficas simples hasta regresiones, cluster, entre otros.

1.1.1 ¿Cuáles son las principales librerías de R y Python?

En R podemos encontrar las librerías `ggplot2`, `tibble`, `tidyr`, `readr`, `purrr`, y `dplyr`. Estas comparten una filosofía propia y se encuentran diseñadas para trabajar naturalmente entre ellas. Se utilizan principalmente para el análisis de datos, al igual que `Caret` para machine learning y `Data.table` para estructura de datos.

En cuanto a Python destacan `matplotlib` y `tensorflow` para la generación de gráficos y cálculos numéricos, respectivamente; al igual que `Scikit-learn`, pensado para la construcción de modelos de aprendizaje automático o machine learning, así como para el análisis de datos.

`Pandas` es muy usada en data science y estadística, `Seaborn` para la visualización de datos estadísticos, y `NumPy` permite generar una estructura de datos universal.

1.1.2 Entorno IDE (entorno de desarrollo integrador) para R-Studio

R cuenta con un IDE propio: R-Studio. Este se puede descargar de la página oficial:

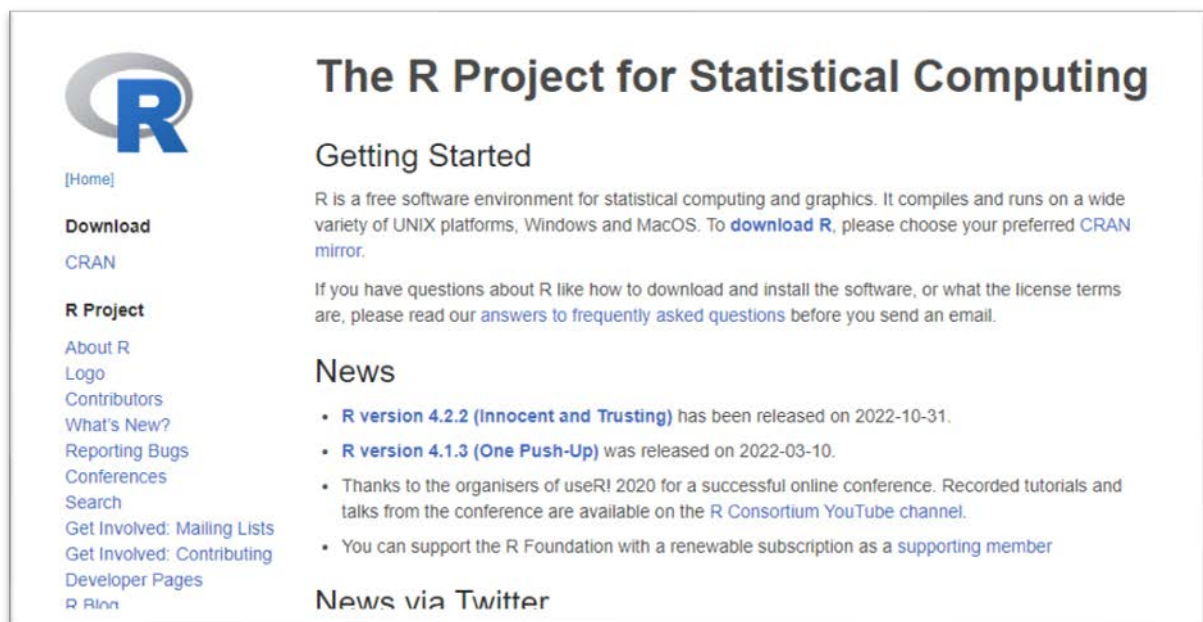


Figura 1. Página oficial. Fuente: [r-project.org](https://www.r-project.org) <https://www.r-project.org>

1.1.3 Primeros pasos en Python

El entorno IDE (entorno de desarrollo integrador) para Python es Pycharm, el que puedes descargar de su sitio oficial <https://www.jetbrains.com/es-es/pycharm/>



Figura 2. Página oficial. Fuente: jetbrains <https://www.jetbrains.com/es-es/pycharm/>

1.1.4 Cómo instalar librerías en RStudio y Pycharm

Podemos hacerlo directamente desde la consola del sistema operativo utilizando el comando pip e install, en el caso de Python. Para RStudio utilizamos el comando `install.packages`.

Ejemplo:

```
>> pip install <<librería>> #para el caso de python
```

```
>> install.packages("nombre de la librería") # para el case de rstudio
```

1.2 Sintaxis del lenguaje RStudio y Python

El término sintaxis hace referencia al conjunto de reglas que definen cómo se tiene que escribir el código en un determinado lenguaje de programación. Es decir, se refiere a la forma en la que debemos escribir las instrucciones para que el computador -o más bien, el lenguaje de programación- nos entienda.

A continuación, se muestran ejemplos de expresiones:

Operaciones aritméticas: suma, resta, multiplicación, división, potenciación, división entera:

$+$, $-$, $*$, $/$, $^$, $\%/\%$ or $\%\%$.

A = b + c /* en lenguaje Python

A <- b + c /* en lenguaje R

Instrucciones de asignación: podemos crear nuevos objetos con el operador de asignación <- R o = en Python. Todas las instrucciones de asignación donde creamos objetos tienen la misma forma:

Nombre_objeto <- valor # en lenguaje R

Nombre_objeto = valor # lenguaje Python

Uso del punto y coma, paréntesis y llaves: este uso permite agrupar sentencias en una misma línea o indicar dónde empieza y termina una instrucción. Por ejemplo:

print("Hola"); print("mundo"); # en lenguaje python y R

x <- 1; y <- 2; z <- 3 # en lenguaje R

x = 1; y = 2; z = 3 # en lenguaje Python

funcionR <- function(par1, par2, ...){ # en lenguaje R

cuerpo de la función

}

def function(): # en lenguaje python

cuerpo de la función

1.3 Principales objetos de estudio para RStudio y Python

Todo el código manipula objetos. El objeto es el concepto principal sobre el cual se fundamenta la tecnología orientada a estos. Un objeto puede ser visto como una entidad que posee atributos y efectúa acciones.

Variables: las variables son tratadas como objetos y sus nombres pueden contener letras, números, puntos y guiones bajos, pero no pueden empezar con un número o un punto seguido de un número. Además, no podemos utilizar las palabras reservadas del lenguaje como if y for.

Ejemplo:

- **a, b, c**
- **Las_variables_pueden_contener_guiones**
- **Las.variables.pueden.contener.puntos**
- **Las_variables_pueden_contener_ounto_guion_y_números**

El tipo de variables de un objeto puede ser int, string, double, float, vector, arreglo, listas y el objeto asume el tipo de variable según el tipo del valor asignado al objeto.

Ejemplo:

```
a <- 1 #el tipo de variable del objeto i es int en lenguaje R  
b = (1,2,3,4) #el tipo de variable del objeto i es un vector en lenguaje Python
```

Objeto vector: los vectores son arreglos ordenados en los cuales se puede almacenar información de tipo numérico (variable cuantitativa), alfanumérico (variable cualitativa) o lógico (TRUE o FALSE), pero no mezclas de estos.

En R, para crear un arreglo, se usa la función `c()` y en Python se usa la función `array()`.

Ejemplo:

```
Vector_r <- c(1,2,3,4,5)
Vector_python = array(1,2,3,4)
```

Objeto matriz: las matrices son arreglos rectangulares de filas y columnas con información numérica, alfanumérica o lógica. Para construir una matriz se usa la función:

```
Matriz_r <- matrix(data=1:20, nrow=4, ncol=5, byrow = FALSE)
Matriz_python = [[1,2,3,4,5],[1,2,3,4,5],[1,2,3,4,5],[1,2,3,4,5]]
```

Objeto lista: una lista es una colección ordenada de cualquier objeto y se crea con la función list() tanto en lenguaje r como Python.

Ejemplo:

```
a <- 1
Vector_r <- c(1,2,3,4,5)
Matriz_r <- matrix(data=1:20, nrow=4, ncol=5, byrow = FALSE)

Lista_r <- list(a, Vector_r, Matriz_r)
Para definir una lista en Python utilizamos los corchetes.
b = (1,2,3,4)
Vector_python = array(1,2,3,4)
Matriz_python = [[1,2,3,4,5],[1,2,3,4,5],[1,2,3,4,5],[1,2,3,4,5]]

Lista_Python = [b, Vector_python, Matriz_python]
```


Objeto función: en Python usamos la palabra reservada llamada def para definir una función:

```
Def funcionPython(arg 1, arg2....arg n): #usa ':' para indicar donde comienza
```

```
Cuerpo de la función #usa indexación para indicar lo que está dentro de la función.
```

En R, la definición de una función se realiza mediante una asignación:

```
Nombre_de_la_funcion <- fuction(arg 1, arg,2 ...arg n){ #usamos '{' para indicar donde  
empieza la función
```

```
Cuerpo de la función
```

```
} # usamos '}' para indicar donde termina la función
```

1.4 Librerías de análisis de datos

La mejor forma de definir qué es una librería es decir que es un código abierto, el que contiene un conjunto de funciones que alguien escribió en alguna parte del mundo y ha puesto a disposición para que cualquiera pueda utilizarlo de forma gratuita.

Por ejemplo, las librerías de Python pueden ser utilizadas para muchas áreas de programación, pues cada sistema cuenta con diferentes procesos que necesitan de una biblioteca específica para resolver diversas funciones.

A continuación, se presentan algunos tipos de librerías de programación, para que elijas las que mejor se acomodan a tus necesidades:

Librerías	Descripción
Visualización	Ideales para entender mejor los datos y tener una mejor comprensión de cualquier problema que pueda surgir.
Cálculo numérico	Las más indicadas para la preparación de datos y cálculo de atributos relevantes.
Machine learning	Están enfocadas en procesar información y en resolver problemas de clasificación y regresión de datos.
Deep learning	Orientadas hacia la predicción de datos, a partir de un conjunto amplio de información.
Inteligencia artificial explicable	Aplican métodos y técnicas tecnológicas que aplican inteligencia artificial para brindar resultados eficientes

Como puedes ver, de acuerdo con el tipo de trabajo que realices, debes seleccionar la librería de Python que más se adecúe a tus objetivos, puesto que no todas te ayudarán a completar las mismas actividades de forma eficiente.

1.4.1 Librerías disponibles para R

A continuación, te mostramos algunas librerías disponibles para R:

Stringr: contiene funciones con las cuales manipular cadenas de caracteres que sirven, por ejemplo, para resolver problemas en la codificación de variables categóricas.



Para conocer esta librería, ingresa a: <https://stringr.tidyverse.org/>

Entre sus muchas funciones, algunas de ellas nos permiten:

- Eliminar espacios en blanco en cualquiera de los lados de las cadenas.

library(stringr)

– **Dividir las cadenas de caracteres según un patrón, pudiendo indicar el número de bloques en los que hacerlo:**

```
fecha <- c(«05/01/2008»)  
  
str_split(string = fecha, pattern = «/»)  
  
> «05» «01» «2008»  
  
str_split_fixed(string = fecha, pattern = «/» , n =2)  
  
> «05» «01/2008»
```

– **Extraer fragmentos de una cadena que corresponden a un determinado patrón:**

```
edad <- c(«20 años», «43 años», «31 años», «25 años»)  
  
patron <- «[0-9]+» #uno o más números  
  
str_extract(string = edad, pattern = patron)  
  
> «20» «43» «31» «25»
```

Lubridate: con esta librería resulta muy sencillo trabajar con fechas y horas. Nos permite convertir cadenas de caracteres al formato de fecha POSIXct, reconociendo distintos formatos.



lubricar

Ingresa a este enlace para saber más: <https://lubridate.tidyverse.org/>

Veamos algunos ejemplos:

library(lubridate)

– **Año, mes y día separados por guiones o barras:**

```
fechas <- c(«2017-05-03», «2017/05/03»)
```

```
ymd(fechas)
```

```
> «2017-05-03» «2017-05-03»
```

– **Día, mes y año sin separación o incluso escritos (en inglés):**

```
fechas <- c(«03052017», «3 May 2017»)
```

```
dmy(fechas)
```

```
> «2017-05-03» «2017-05-03»
```

– **Fechas y horas, pudiendo indicar la zona horaria:**

```
ymd_hms(«2017-05-03 09:00:00», tz = «Europe/Madrid»)
```

```
> «2017-05-03 09:00:00 CEST»
```

Tidyr y dplyr: gracias a estas librerías podemos manipular los datos cuando estos se disponen en forma de tablas.

Tidyr: proporciona diversas funciones con las que podemos convertir las filas en columnas y viceversa, dividir una columna en varias o juntar distintas columnas en una.

library(tidyr)

Por ejemplo, imaginemos que tenemos una columna en nuestros datos como la siguiente:

medida <fctr>
2.3 cm
19 mm
2.2 cm
1.5 cm

Podemos separarla en dos columnas, una para el valor de la medida y otra para las unidades, y eliminar la columna original.

```
separate(data = datos, col = medida, into = c(«longitud», «unidades»), sep = » «,  
remove = TRUE)
```

longitud <chr>	Unidades <chr>
2.3	cm
19	mm
2.2	cm
1.5	cm



Para saber más sobre **Tidyr**, ingresa a este enlace: <https://tidyr.tidyverse.org/>

Dplyr: dispone de funciones mediante las cuales seleccionar las columnas que nos interesan según diversos criterios (letra con la que comienza el nombre de la variable, patrón contenido en este, etc.), filtrar las filas y solo tomar aquellas para las cuales los valores de una o más variables cumplen determinadas condiciones, ordenar las filas y agruparlas, en función del valor que tomen para una determinada variable, o crear nuevas variables a partir de las existentes. Además, podemos calcular fácilmente datos estadísticos como la media o la varianza de cada una de las variables.

A modo de ejemplo, con el conjunto de datos iris disponible en R, que cuenta con 150 medidas de 3 especies diferentes de iris, filtramos las filas, conservando aquellas en las que la longitud de los sépalos es superior a 5 y las ordenamos por dicha longitud: `library(dplyr)`.



Conoce más sobre **Dplyr**, accediendo aquí: <https://dplyr.tidyverse.org/>

Readr: contiene funciones que permiten cargar y manipular una serie de datos.

- Eliminar lectura de un archivo en formato CSV.



Conoce más sobre Readr, en este enlace: <https://readr.tidyverse.org/>

ggplot2: para visualizar los datos, ggplot2 nos ofrece la posibilidad de crear una gran cantidad de gráficos, como scatter plots, gráficos de barras, boxplots o histogramas, entre otros muchos. Se basa en añadir capas en las que se especifican los distintos parámetros del gráfico: datos por representar, colores, formas geométricas, transformaciones de tipo estadístico aplicadas a los datos, escala en cada uno de los ejes o sistema de coordenadas.



Para conocer todo sobre **ggplot2**, ingresa a este enlace: <https://ggplot2.tidyverse.org/>

Por ejemplo, para el conjunto de datos anterior, representamos un boxplot de la longitud de los pétalos para cada una de las especies de iris: `library(ggplot2)`.

```
ggplot(datos, aes(x = Species, y = Petal.Length)) + geom_boxplot()
```

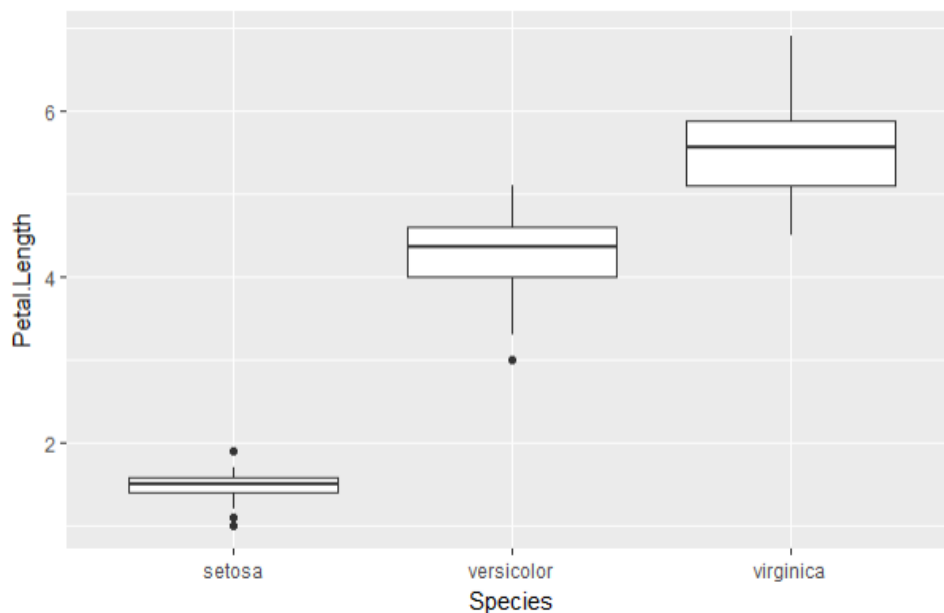


Figura 3. Resultado librería ggplot2. Fuente: elaboración propia.

1.4.2 Librerías disponibles para Python

Existen muchas librerías de Python que mejoran la visualización de datos de sus programadores. A continuación, te mostramos algunas librerías disponibles para Python:

Matplotlib: esta es una de las librerías más usadas, pues su procesamiento y generador de gráficos son de amplia calidad. Además, da la facilidad de publicar estos datos de manera física y bajo impresión, como también digitalmente.

Utiliza trazados 2D y muchos tipos de gráficos que se crean con pocas líneas de código. Algunos ejemplos son los histogramas, diagramas de barras, espectros de potencia, series temporales, diagramas de errores, entre otros.

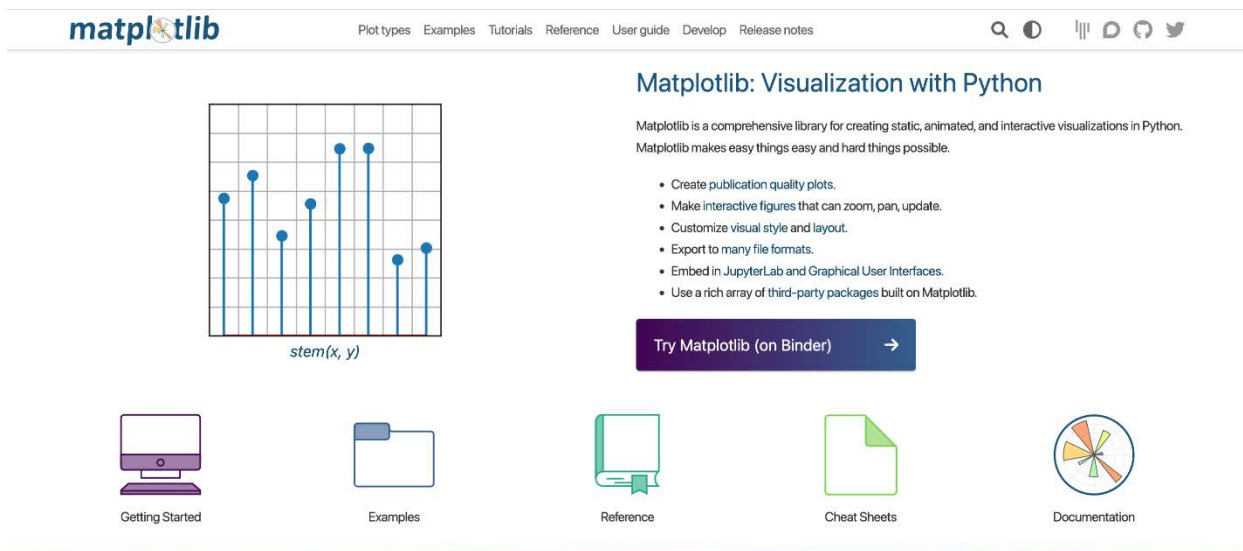


Figura 4. Pantalla de inicio de Matplotlib. Fuente: <https://matplotlib.org/>

Seaborn: es una librería gráfica de Python especializada en la visualización de datos estadísticos. Dentro de sus principales características, podemos destacar su interfaz de alto nivel, la cual es atractiva y muy interactiva. Este aspecto es fundamental cuando llega el momento de esclarecer los datos luego de un análisis, pues hace que la visualización de estos sea más sencilla. Asimismo, es compatible con otras librerías de Python como Pandas y Matplotlib.

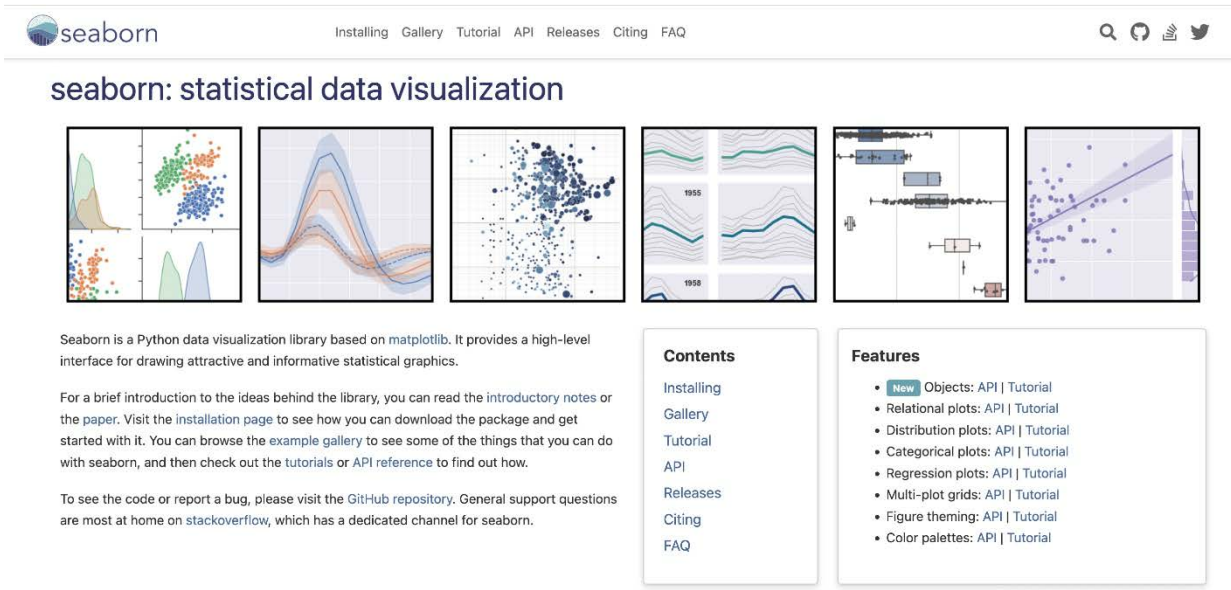


Figura 5. Pantalla de inicio de Seaborn. Fuente: <https://seaborn.pydata.org/>

NumPy: de todas las librerías de Python, Numpy te permite crear una estructura universal de datos para facilitar su análisis e intercambio de varios algoritmos. Al igual que varias librerías de Python, esta implementa vectores multidimensionales y matrices que almacenan una gran cantidad. A su vez, posee funciones matemáticas de alto nivel y utiliza diversas estructuras de datos.

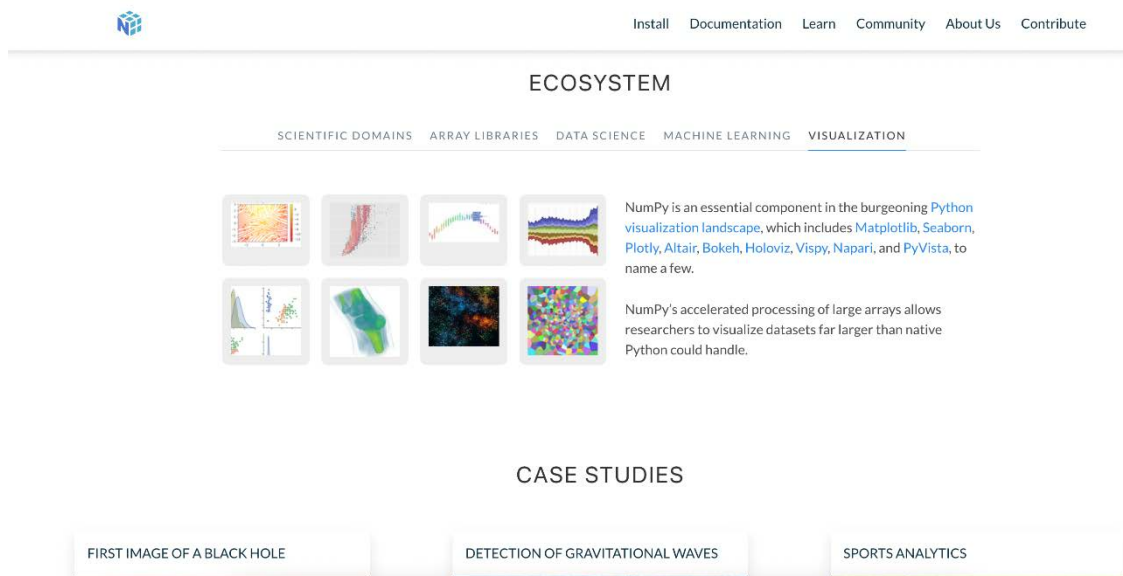


Figura 6. Pantalla de inicio de NumPy. Fuente: <https://numpy.org/>

Pandas: si estás buscando una biblioteca de Python enfocada en los datos científicos, Pandas es una de las más útiles que encontrarás. Sus principales funciones se conforman por series de datos y el data frame para dos dimensiones. Adicionalmente, es muy sencilla de usar para manipular datos y se utiliza en campos como finanzas, ciencias sociales, estadística e ingeniería.

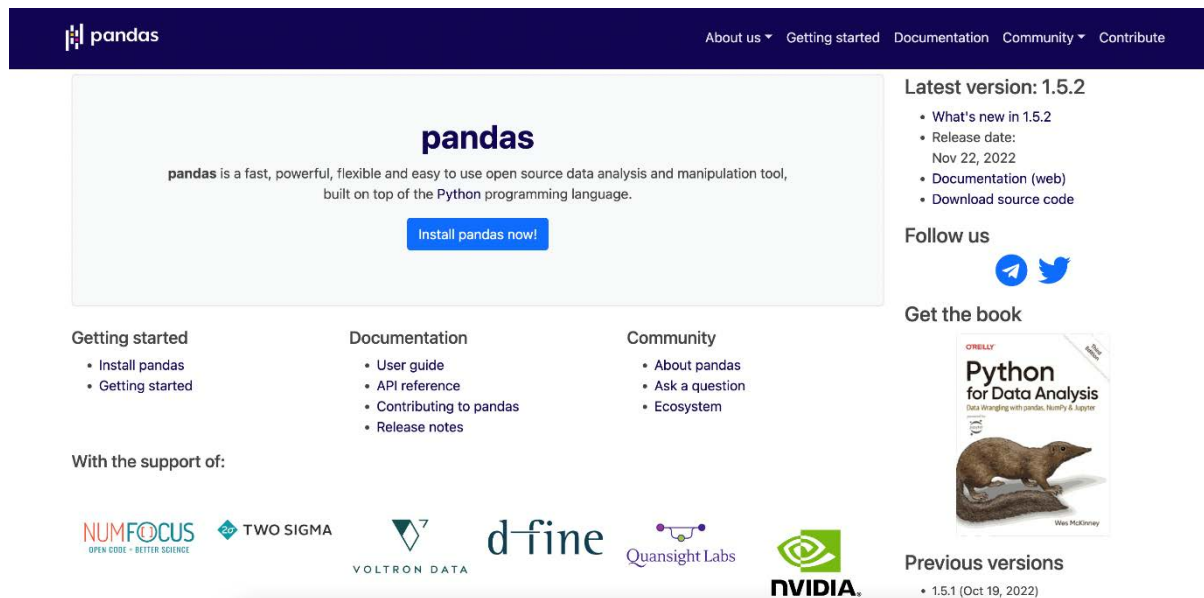


Figura 7. Imagen de la pantalla de inicio de Pandas. Fuente: <https://pandas.pydata.org/>

Scikit-learn: esta librería de Python cuenta con funciones de machine learning y análisis de datos. Es una herramienta muy fácil de usar gracias a sus múltiples técnicas de aprendizaje automático.

Puedes utilizar Scikit-Learn para resolver problemas de clasificación y de regresión, tales como máquinas de vectores de soporte, árboles de decisión, regresión lineal y polinómica, clustering, reducción de dimensionalidad, y otros. Su interfaz es simple y consistente, porque se puede moldear solo con una línea de código.

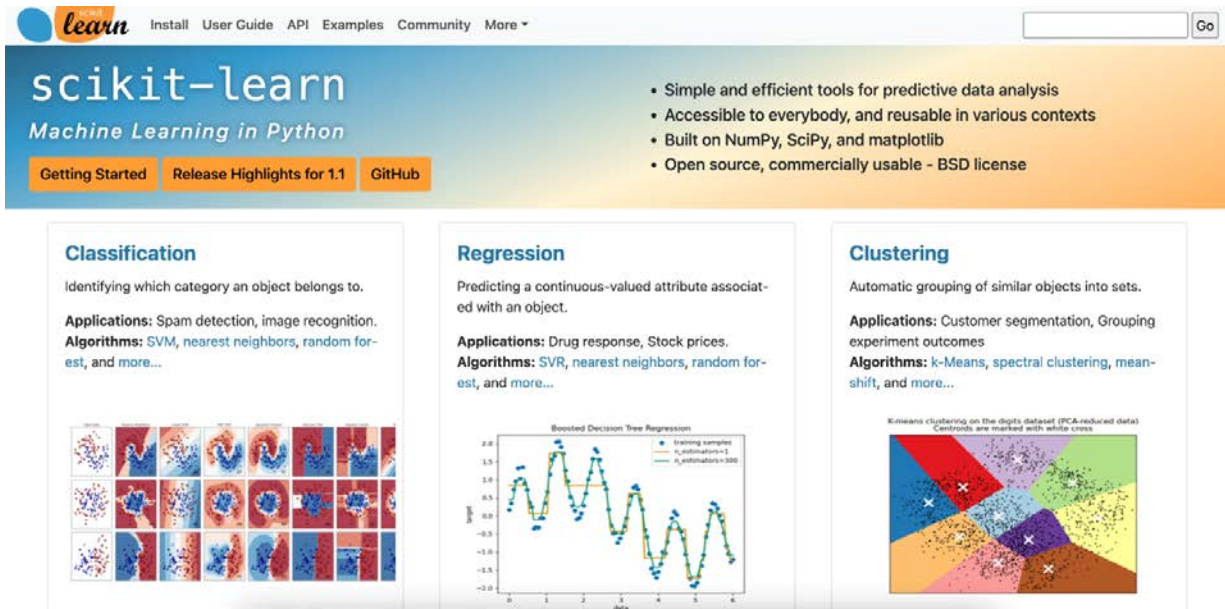


Figura 8. Imagen de la pantalla de inicio de Scikit-learn. Fuente: <https://scikit-learn.org/stable/>

TensorFlow: forma parte de las librerías de Python que fueron desarrolladas por Google. Permite realizar cálculos numéricos gracias a diagramas de flujo de datos y se usa para varias aplicaciones de cálculo científico, como Deep Learning.

Según la página oficial de TensorFlow, esta es una “plataforma de código abierto para el aprendizaje automático que cuenta con un ecosistema integral y flexible de herramientas, bibliotecas y recursos, la cual permite que los investigadores innoven con el aprendizaje automático y los desarrolladores creen aplicaciones con tecnología de AA fácilmente”.



Figura 9. Imagen de la pantalla de inicio de TensorFlow. Fuente: <https://www.tensorflow.org/?hl=es-419>

2. VISUALIZACIÓN Y ANALÍTICA DE DATOS

2.1 Carga y visualización de datos

En la práctica, cuando hablamos de visualización de datos necesitamos tener claros los siguientes conceptos:

Base de datos: es un repositorio de información que contiene una gran cantidad de datos, relacionados entre sí y estructurados en tablas o también llamadas entidades. Estas cambian en el tiempo y son administradas por un software llamado motor de base de datos. Estos administran su funcionamiento y otorgan los accesos a las distintas necesidades como desarrollos de sistemas de información, software de paquetes como ERP, administrativos, y otros. Por ejemplo:

- **Registros de transacciones:** las operaciones realizadas con una tarjeta de crédito, así como las llamadas realizadas con un celular u otro tipo de transacciones comerciales cotidianas, generan todo un conjunto de registros que llegan a una base de datos de la empresa.
- **Historial médico:** cada vez que acudimos al doctor o a un hospital, se actualiza la información respecto a nuestra salud, al tratamiento recibido y demás detalles médicos, en un archivo que lleva registro de nuestra historia médica. Ello para en caso que luego se requiera conocer datos específicos, como operaciones o tratamientos recibidos.

Set de datos: es un conjunto de datos tabulados que es estático en el tiempo, sin tener modificaciones. Son resultados de consultas específicas a una base de datos o, simplemente, son el resultado de procesos de recolección de información en un tiempo determinado y son usados para realizar análisis. Por ejemplo:

- **Encuestas:** información del resultado de un proceso de encuestas en particular.
- **Información científica:** información de las mediciones de un experimento o procedimiento científico, como también información del resultado de observaciones astronómicas.
- **Consulta a una base de datos:** es el resultado de una consulta específica a un conjunto de tablas de una base de datos de algún sistema de información.

Ambos métodos de almacenamiento de información son fuentes de información para el estudio de la analítica de datos.

2.1.1 Visualización de datos usando data frame de la librería Pandas de Python

Para este taller usaremos un set de datos en formato CSV. Además, aplicaremos los principales pasos o buenas prácticas de visualización de datos.

Primero, debemos importar la información por analizar. Para eso, usaremos la librería Pandas para Python (para r se usa la librería readr).

Ejemplo 1: carga de archivo csv

```
Import pandas as pd #incluir librería librería pandas
df = pd.read_csv('C:/../archivo_data.csv') # cargando set de datos en formato csv.

df.info() # nos muestra la cantidad de columnas y su tipo.

df.dtypes # nos muestra los objetos que componen las columnas.

import pandas as pd
df = pd.read_csv('c:/../ejemplo_data.csv')
df.info()
```

```
<class                                'pandas.core.frame.DataFrame'>
RangeIndex: 6          entries, 0      to 5
Data      columns (total 8      columns):
#      Column      Non-Null Count  Dtype
---  -
0      ID          6 non-null    int64
1      Nombre      6 non-null    object
2      2016         6 non-null    object
3      2017         6 non-null    object
4      Crecimiento  6 non-null    object
5      Unidades     6 non-null    object
6      fecha        6 non-null    object
7      Activo       6 non-null    object
dtypes: int64(1), object(7)
memory usage: 512.0+ bytes
```

Ejemplo 2: revisión de los tipos de datos cargados

df.dtypes	
ID	int64
Nombre	object
2016	object
2017	object
Crecimiento	object
Unidades	object
fecha	object
Activo	object
dtype: object	
df	

	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
0	10002	Verde Mar	\$125,000.00	\$162500.00	30.00%	500	1-10-2015	1
1	552278	Manantial S.A.	\$920,000.00	\$101,2000.00	10.00%	700	6-23-2014	0
2	23477	ACME	\$50,000.00	62500.00	25.00%	125	3-12-2016	1
3	24900	Andes sur	\$350,000.00	490000.00	4.00%	75	10-28-2015	1
4	651029	San Pablo	\$15,000.00	\$12750.00	-15.00%	No	2-15-2014	0
5	651029	San Pablo	-\$15,000.00	\$ 12750.00	-15.00%	No	2-15-2014	n

Ejemplo 3: uso de la librería matemáticas para hacer cálculo estadísticos o matemáticos, para comprender el comportamiento de los valores del data set.

```

import random
Id_Alumno = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20]
NOMBRES = ['Nicolas','Alonso','Juan','Carolina','Humberto', 'Malva','Claudia','Patricia','Felipe','Cristian']
l1 = []
for n in range(20):
    l1.append(random.choice(NOMBRES))
pd.Series(l1)
l2 = []
for n in range(20):
    l2.append(random.randint(18,24))
pd.Series(l2)

```

```

l3 = []
for n in range(20):
    l3.append(random.uniform(1,7))
pd.Series(l3)
PESOS_NOTA
[1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20,1/20]

d = dict()
d = {'Id_alumno' : pd.Series(Id_Alumno),'Nombre' : pd.Series(l1), 'Edad' : pd.Series(l2), 'Nota' :
pd.Series(l3), 'Ponderacion' : pd.Series(PESOS_NOTA) }
df = pd.DataFrame(d)

```

	Id_alumno	Nombre	Edad	Nota	Ponderación
0	1	Claudia	23	3.509351	0.05
1	2	Juan	21	3.160651	0.05
2	3	Alonso	21	5.262787	0.05
3	4	Patricia	20	4.298663	0.05
4	5	Nicolas	18	3.855920	0.05
5	6	Juan	19	4.700876	0.05
6	7	Felipe	21	4.752685	0.05
7	8	Carolina	18	3.538994	0.05
8	9	Alonso	18	3.368099	0.05
9	10	Juan	21	3.892812	0.05
10	11	Malva	21	2.311373	0.05
11	12	Malva	22	2.854839	0.05
12	13	Alonso	20	1.955521	0.05
13	14	Carolina	18	1.029374	0.05
14	15	Nicolas	19	6.141772	0.05
15	16	Malva	23	5.934080	0.05
16	17	Patricia	20	2.698209	0.05
17	18	Humberto	18	5.680674	0.05
18	19	Malva	18	1.536414	0.05
19	20	Claudia	19	5.478843	0.05

```
df["Edad"].mean()
Resultado: 21.25
```

```
import numpy as np
np.average(df['Nota'], weights = df['Ponderacion'])
df["Nota"].mean()
Resultado: 4.110174991248155
```

```
np.round(df["Nota"].max()-df["Nota"].min(),2)
Resultado: 5.84
```

```
np.round(df["Edad"].var(),2)
Resultado: 5.36
```

```
np.round(df["Nota"].std(),2)
Resultado: 1.89
```

```
df[['Edad', 'Nota']].quantile([0.25,0.5,0.75])
```

Resultado:

	Edad	Nota
0.25	19.0	2.363054
0.50	21.0	4.541626
0.75	24.0	5.409660

2.2 Reemplazos de valores y mutación de datos

Es importante que la información esté consistente y con los formatos correctos, para que no se encuentren indeterminaciones en las operaciones que se aplican en el proceso de analítica de datos.

Ejemplo 1: eliminación de datos con errores como valores nulos

```
df["Activo"] = pd.to_numeric(df['Activo'], errors = 'coerce').fillna(0)
df["Activo"] = df['Activo'].astype('Bool')
df
```


	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
0	10002	Verde Mar	\$125,000.00	\$162500.00	30.00%	500	1-10-2015	True
1	552278	Manantial S.A.	\$920,000.00	\$101,2000.00	10.00%	700	6-23-2014	False
2	23477	ACME	\$50,000.00	62500.00	25.00%	125	3-12-2016	True
3	24900	Andes sur	\$350,000.00	490000.00	4.00%	75	10-28-2015	True
4	651029	San Pablo	\$15,000.00	\$12750.00	-15.00%	No	2-15-2014	False
5	651029	San Pablo	-\$15,000.00	\$ 12750.00	-15.00%	No	2-15-2014	False

Ejemplo 2: formatear valores a un tipo de variable de objeto.

```
df["Unidades"] = pd.to_numeric(df['Unidades'], errors = 'coerce').fillna(0)
df["Unidades"] = df['Unidades'].astype('int')
df
```

	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
0	10002	Verde Mar	\$125,000.00	\$162500.00	30.00%	500	1-10-2015	True
1	552278	Manantial S.A.	\$920,000.00	\$101,2000.00	10.00%	700	6-23-2014	False
2	23477	ACME	\$50,000.00	62500.00	25.00%	125	3-12-2016	True
3	24900	Andes sur	\$350,000.00	490000.00	4.00%	75	10-28-2015	True
4	651029	San Pablo	\$15,000.00	\$12750.00	-15.00%	0	2-15-2014	False
5	651029	San Pablo	-\$15,000.00	\$ 12750.00	-15.00%	0	2-15-2014	False

Ejemplo 3: utilizar funciones para reemplazar valores.

```
def elimina_comas_pesos(val):
    nuevo = val.replace(',','').replace('$','')
    return float(nuevo)
def elimina_porcentaje(val):
    nuevo = val.replace('%','')
    return float(nuevo)/100
df["2016"] = df['2016'].apply(elimina_comas_pesos)
df["Crecimiento"] = df['Crecimiento'].apply(elimina_porcentaje)
df
```

	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
0	10002	Verde Mar	125000.0	\$162500.00	0.30	500	1-10-2015	True
1	552278	Manantial S.A.	920000.0	\$101,2000.00	0.10	700	6-23-2014	False
2	23477	ACME	50000.0	62500.00	0.25	125	3-12-2016	True
3	24900	Andes sur	350000.0	490000.00	0.04	75	10-28-2015	True
4	651029	San Pablo	15000.0	\$12750.00	-0.15	0	2-15-2014	False
5	651029	San Pablo	-15000.0	\$ 12750.00	-0.15	0	2-15-2014	False

2.3 Ordenamiento y filtros

A veces ayuda mucho ordenar nuestro conjunto de datos, con el objetivo de obtener una mejor visualización como, por ejemplo, estudiar si ciertos tipos de valores son discretos o continuos, si existen intervalos indeterminados, o visualizar rápidamente si existen puntos de inflexión, o determinar si hay algún patrón en común con algunos valores.

Ejemplo 1: ordenamiento por una columna del data set.

```
df = df.sort_values(by="Nombre")
df
```

	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
2	23477	ACME	50000.0	62500.00	0.25	125	3-12-2016	True
3	24900	Andes sur	350000.0	490000.00	0.04	75	10-28-2015	True
1	552278	Manantial S.A.	920000.0	\$101,2000.00	0.10	700	6-23-2014	False
4	651029	San Pablo	15000.0	\$12750.00	-0.15	0	2-15-2014	False
5	651029	San Pablo	-15000.0	\$ 12750.00	-0.15	0	2-15-2014	False
0	10002	Verde Mar	125000.0	\$162500.00	0.30	500	1-10-2015	True

Ejemplo 2: filtrar por un conjunto de datos del data set.

```
df_mascara = df['Activo']==True
df_filtrado = df[df_mascara]
print(df_filtrado)
```

Salida de la instrucción:

	ID	Nombre	2016	2017	Crecimiento	Unidades	fecha	Activo
2	23477	ACME	50000.0	62500.00	0.25	125	3-12-2016	True
3	24900	Andes sur	350000.0	490000.00	0.04	75	10-28-2015	True
0	10002	Verde Mar	125000.0	\$162500.00	0.30	500	1-10-2015	True

2.4 Gráfica

La gráfica es parte fundamental de la visualización de datos, pero también es una excelente herramienta de apoyo para el proceso de analítica de datos. Nos permite ver el comportamiento de la información desde lo micro a lo macro, “small data a big data”. Así, es posible detectar, a simple vista y de forma rápida, fenómenos, tendencias y patrones.

Ejemplo: gráfica de funciones matemáticas

```
import matplotlib.pyplot as plt
def f1(x):
    return(2*(x**2)+5*x-2)
def f2(x):
    return(4*x+1)
x=range(-10,15)
plt.plot(x,[f1(i) for i in x])
plt.plot(x,[f2(i) for i in x])
```

[<matplotlib.lines.Line2D at 0x2310365bbe0>]

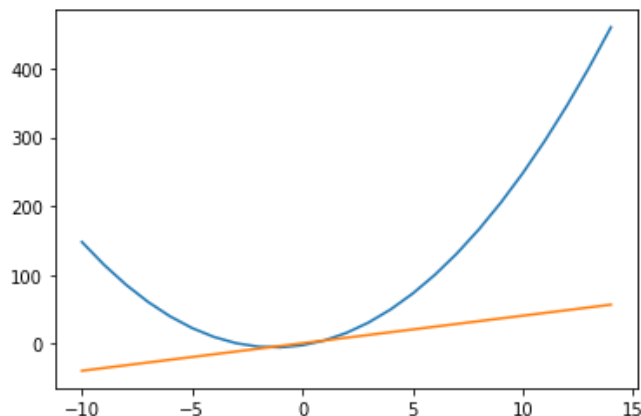


Figura 10. Resultado librería Matplotlib. Fuente: elaboración propia.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

np.random.seed(4) ##valor de semilla
X = 3*np.random.rand(100,1) # Crear 100 muestras de variable aleatoria de para la variable dependiente
entre 0 y 3
y = 5 + 2*X - 2.5*np.random.randn(100,1) # Crear 100 muestras para la variable dependiente de acuerdo
a 5 + 2X -2.5*uniforme(0,1)

## gráfico de dispersión
plt.plot(X,y, "b.")
plt.ylabel('y')
plt.xlabel('X')
plt.axis([0, 3, 0, 15])
plt.legend(["Gráfico de dispersión"])
plt.show()
```

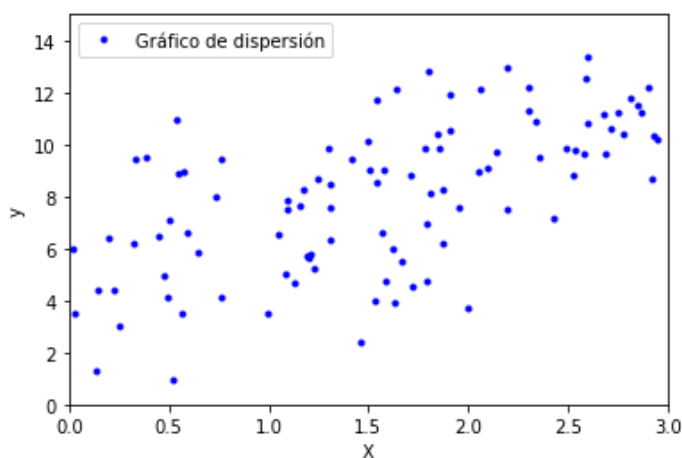


Figura 11. Resultado librería Matplotlib. Fuente: elaboración propia.

```

X_ = np.c_[np.ones((100,1)), X] #concatenar 1 a variables independientes
theta_opt = np.linalg.inv(X_.T.dot(X_)).dot(X_.T).dot(y) #calcular thetas de forma cerrada
theta_opt ## error cuadrado medio cerrado

## predicción
X_nuevo = np.array([[0], [3]])
print(X_nuevo)
X_nuevo_ = np.c_[np.ones((2,1)), X_nuevo]
print(X_nuevo_)
y_pred = X_nuevo_.dot(theta_opt)
print('prediccion', y_pred)

##gráfico con la predicción
plt.plot(X_nuevo, y_pred, 'r')
plt.plot(X,y, "b.")
plt.ylabel('y')
plt.xlabel('X')
plt.axis([0, 3, 0, 15])
plt.legend(["Regresión Lineal"])
plt.show()

```

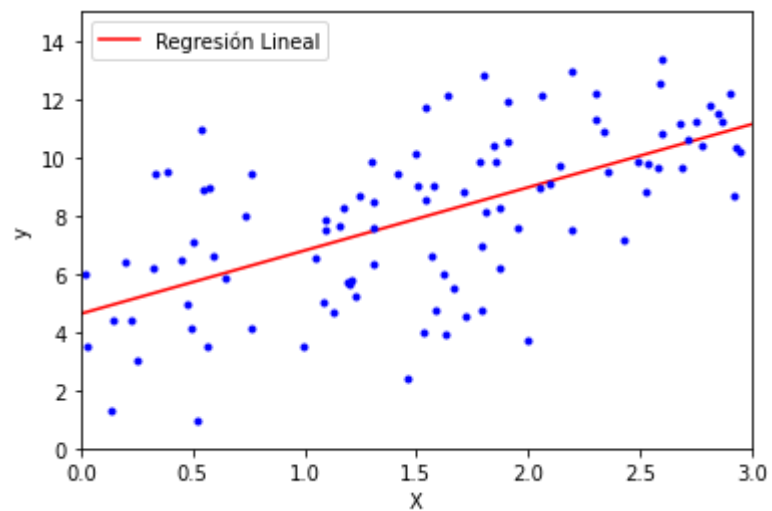


Figura 12. Resultado librería Matplotlib. Fuente: elaboración propia.

2.5 Modelos matemáticos

Regresión lineal: el algoritmo de regresión lineal consiste en determinar la mejor curva que representa la dependencia entre la variable dependiente con la independiente, para un conjunto de datos determinados.

El modelo se basa en buscar los mejores estimadores del modelo C_0 y C_1 .

$$Y = C_0 + C_1 x_1 + e$$

Donde Y es la variable dependiente, C_0 y C_1 son los estimadores del modelo que corresponde al interceptor y pendiente de la curva, x_1 son los descriptores o variables, e es el error aleatorio.

Bajo este modelo, lo que se busca es encontrar los mejores estimadores del modelo.

Regresión lineal con múltiples descriptores:

$$Y = C_0 + C_1 x_1 + \dots + C_i x_i + \dots + C_n x_n$$

Y = predicción

N = número de descriptores.

X_i = valor i enésimo del descriptor.

C_i = i enésimo parámetro del modelo

K-means: es un algoritmo de clasificación no supervisada (clusterización) que agrupa objetos en k grupos, basándose en sus características. El agrupamiento se realiza minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster. Se suele usar la distancia cuadrática.

El algoritmo consta de tres pasos:

1. **Inicialización:** una vez escogido el número de grupos, k , se establecen k centroides en el espacio de los datos, por ejemplo, escogiendo aleatoriamente.
2. **Asignación objetos a los centroides:** cada objeto de los datos es asignado a su centroide más cercano.

3. **Actualización centroides:** se actualiza la posición del centroide de cada grupo, tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo.

Se repiten los pasos 2 y 3 hasta que los centroides no se mueven, o se mueven por debajo de una distancia umbral en cada paso.

$$\mathbf{m}_c = \frac{1}{N_c} \sum_{i=1}^{N_c} \mathbf{x}_i$$

Donde **mc** es el número de centroide.

- **Ejemplo 1:** algoritmos que aprenden desde una base de datos que contiene características de clientes como edad, lugar donde vive y nivel de ingreso, junto con entregar una predicción de tipo de consumo del individuo.
- **Ejemplo 2:** algoritmos que aprenden desde una base de datos que contienen patrones, la que es capaz de entregar una salida relacionada como, por ejemplo, reconocimiento facial, clasificaciones de estrellas en el espacio.
- **Ejemplo 3:** algoritmo que aprenden de información que provienen de resultados de procedimientos, experimentos, juegos de azar, ajedrez y predicen escenarios posibles para tomar una decisión o políticas en una empresa como reposición de inventario, pedir, cuándo pedir y cuánto pedir.

2.6 Introducción a los modelos de inteligencia artificial (AI)

Conceptos fundamentales de aprendizaje supervisado y no supervisado.

Aprendizaje de máquina: podría definirse como un conjunto de métodos que detectan automáticamente patrones complejos en los datos, dichos patrones pueden ser usados para predecir el futuro. El aprendizaje de máquina se basa en un modelo que se compone de la experiencia que nos entrega la analítica de datos, una tarea que es un objetivo que se desea alcanzar según la experiencia y el rendimiento que nos dicen qué tan eficiente es el modelo.

Ejemplo:

- **Experiencia:** datos de la compra de clientes, imágenes médicas con diagnósticos.
- **Tarea:** agrupar clientes de acuerdo con preferencia, predecir un diagnóstico.
- **Rendimiento:** similitud dentro de los grupos de clientes, error en la predicción del diagnóstico.

Tipos de aprendizaje de máquinas:

- **Aprendizaje supervisado:** permite predecir si el comportamiento de un cliente será bien pagado o no, o predecir el puntaje de logro en una prueba estandarizada.
- **Aprendizaje no supervisado:** permite detectar segmentos de clientes y generar una estrategia de marketing.
- **Aprendizaje reforzado:** permite decisiones en tiempo real como, por ejemplo, pedir o no pedir reposición de productos en un supermercado, o definir los movimientos de un robot bajo ciertas variables de ambiente y contexto.

Referencias bibliográficas

Barros, O. (2003). *Rediseño de procesos de negocios mediante el uso de patrones*. Dolmen Ediciones.

Barros, O. (2016). *Ingeniería de negocios: diseño integrado de servicios, sus procesos y apoyo TI*. MBA Ingeniería Industrial, Universidad de Chile.

Carrasco, J. B. (2009). *Gestión de procesos*. Santiago-Chile: Editorial Evolución.

Géron, A. (2020). *Aprende machine learning con Scikit-Learn, Keras y Tensorflow. Conceptos, herramientas y técnicas para construir sistemas inteligentes*. Anaya.

Martínez, A. & Cegarra, J. (2014). *Gestión por procesos de negocio: organización horizontal*. Ecobook - Editorial del Economista.

Martínez, E. (2018). *Iniciación en la economía y gestión de empresas*. Universidad Politécnica de Cartagena.

Marqués, F. (2017). *R en profundidad. Programación, gráficos y estadística*. Alfaomega.

Reinoso, J. F. & Uribe, M. (2014). *Indicadores de gestión*. Ediciones de la U.

Sandoval, P. (2021). *¿Cómo utilizar Python?* [Apunte docente]. Universidad Andrés Bello, Santiago.7

Wilkinson, C. (2020). *Ciencia de datos Python: Guía definitiva para que los principiantes aprendan los fundamentos de la ciencia de datos con Python*. Independently Published.